

Operating Systems

Sheet 11 Solutions

IO Management, Disk Scheduling, RAID, and File Management

Name: Yehia Said Gewily

ID: 21010705

Date: May 2026

1 IO Management and Disk Scheduling

1. Disk Scheduling Head Movements

Given request queue:

26, 37, 100, 14, 88, 33, 99, 12

Initial head position = 26.

a. FCFS

FCFS services requests in the same order as given.

$26 \rightarrow 37 \rightarrow 100 \rightarrow 14 \rightarrow 88 \rightarrow 33 \rightarrow 99 \rightarrow 12$

$$\begin{aligned} &|37 - 26| + |100 - 37| + |14 - 100| + |88 - 14| + |33 - 88| + |99 - 33| + |12 - 99| \\ &= 11 + 63 + 86 + 74 + 55 + 66 + 87 = \boxed{442 \text{ cylinders}} \end{aligned}$$

b. SSTF

SSTF always chooses the closest pending request to the current head position.

$26 \rightarrow 33 \rightarrow 37 \rightarrow 14 \rightarrow 12 \rightarrow 88 \rightarrow 99 \rightarrow 100$

$$\begin{aligned} &= |33 - 26| + |37 - 33| + |14 - 37| + |12 - 14| + |88 - 12| + |99 - 88| + |100 - 99| \\ &= 7 + 4 + 23 + 2 + 76 + 11 + 1 = \boxed{124 \text{ cylinders}} \end{aligned}$$

c. SCAN, going up

The sheet does not specify the maximum cylinder number. For SCAN and C-SCAN, we assume the common disk range:

0 to 199

Going up first, the head services higher requests, reaches the end, then reverses.

$26 \rightarrow 33 \rightarrow 37 \rightarrow 88 \rightarrow 99 \rightarrow 100 \rightarrow 199 \rightarrow 14 \rightarrow 12$

Total movement:

$$(199 - 26) + (199 - 12) = 173 + 187 = \boxed{360 \text{ cylinders}}$$

If the instructor intended LOOK instead of full SCAN, then the head stops at the highest request 100 and reverses:

$26 \rightarrow 33 \rightarrow 37 \rightarrow 88 \rightarrow 99 \rightarrow 100 \rightarrow 14 \rightarrow 12$

$$(100 - 26) + (100 - 12) = 74 + 88 = \boxed{162 \text{ cylinders}}$$

d. C-SCAN, going up

C-SCAN services upward requests, goes to the end, jumps back to the beginning, then continues upward.

$26 \rightarrow 33 \rightarrow 37 \rightarrow 88 \rightarrow 99 \rightarrow 100 \rightarrow 199 \rightarrow 0 \rightarrow 12 \rightarrow 14$

Total movement:

$$(199 - 26) + (199 - 0) + (14 - 0) = 173 + 199 + 14 = \boxed{386 \text{ cylinders}}$$

If using C-LOOK instead, without moving to the physical disk ends:

$26 \rightarrow 33 \rightarrow 37 \rightarrow 88 \rightarrow 99 \rightarrow 100 \rightarrow 12 \rightarrow 14$

$$(100 - 26) + (100 - 12) + (14 - 12) = 74 + 88 + 2 = \boxed{164 \text{ cylinders}}$$

2. File Access Time for Contiguous and Indexed Allocation

Given:

- 8 sectors per track.
- 512 bytes per sector.
- Rotational speed = 3000 rpm.

- Average seek time = 15 ms.
- File size = 8 blocks.

Assume one block = one sector.

Rotational speed:

$$3000 \text{ rpm} = \frac{3000}{60} = 50 \text{ revolutions/sec}$$

Time for one revolution:

$$\frac{1}{50} = 0.02 \text{ sec} = 20 \text{ ms}$$

Average rotational delay:

$$\frac{20}{2} = 10 \text{ ms}$$

Transfer time for one sector:

$$\frac{20}{8} = 2.5 \text{ ms}$$

a. Contiguous allocation

The 8 file blocks are stored next to each other. Since there are 8 sectors per track, the whole file can be read in one full rotation after reaching the correct track.

$T = \text{seek} + \text{average rotational delay} + \text{transfer of 8 sectors}$

$$T = 15 + 10 + 20 = \boxed{45 \text{ ms}}$$

b. Indexed allocation, not contiguous

We first read the index block, then read the 8 data blocks. Since the data blocks are not contiguous, each block may require a separate average seek and rotational delay.

Number of blocks accessed:

$$1 \text{ index block} + 8 \text{ data blocks} = 9 \text{ blocks}$$

Time per block:

$$15 + 10 + 2.5 = 27.5 \text{ ms}$$

Total time:

$$9 \times 27.5 = \boxed{247.5 \text{ ms}}$$

3. Disk Transfer and Access Time

Given:

- Rotational speed = 15,000 rpm.
- Sector size = 512 bytes.
- 400 sectors per track.
- 1000 tracks.
- Average seek time = 4 ms.
- File size = 1 MByte.

Assume:

$$1 \text{ MByte} = 1 \text{ MiB} = 2^{20} = 1,048,576 \text{ bytes}$$

Rotational speed:

$$15000 \text{ rpm} = \frac{15000}{60} = 250 \text{ rev/sec}$$

Time for one revolution:

$$\frac{1}{250} = 0.004 \text{ sec} = 4 \text{ ms}$$

Average rotational delay:

$$\frac{4}{2} = \boxed{2 \text{ ms}}$$

Transfer time for one sector:

$$\frac{4}{400} = 0.01 \text{ ms}$$

Number of sectors in the file:

$$\frac{1,048,576}{512} = 2048 \text{ sectors}$$

a. Transfer time for the file

$$2048 \times 0.01 = \boxed{20.48 \text{ ms}}$$

b. Average access time for this file

$T = \text{average seek} + \text{average rotational delay} + \text{transfer time}$

$$T = 4 + 2 + 20.48 = \boxed{26.48 \text{ ms}}$$

c. Rotational delay

$$\boxed{2 \text{ ms}}$$

d. Total time to read 1 sector

$$T = 4 + 2 + 0.01 = \boxed{6.01 \text{ ms}}$$

e. Total time to read 1 track

One track contains 400 sectors, which equals one full rotation.

Transfer time for one track = 4 ms

$$T = 4 + 2 + 4 = \boxed{10 \text{ ms}}$$

4. Magnetic Disk Capacity

Given:

- 5 platters.
- 2048 tracks per platter.
- 1024 sectors per track.
- 512 bytes per sector.

Total capacity:

$$\begin{aligned} &5 \times 2048 \times 1024 \times 512 \\ &= 5,368,709,120 \text{ bytes} \end{aligned}$$

Since:

$$\begin{aligned} 1 \text{ GiB} &= 2^{30} = 1,073,741,824 \text{ bytes} \\ \frac{5,368,709,120}{1,073,741,824} &= \boxed{5 \text{ GiB}} \end{aligned}$$

Note: If the question intended 2048 tracks per surface and each platter has two usable surfaces, then the capacity would be doubled to 10 GiB.

5. Is Striping Necessary for Improving I/O Request Rate?

Disk striping improves **data transfer rate** when one large I/O request is split across multiple disks, because the disks transfer different parts of the same request in parallel.

However, for **I/O request rate**, striping is not always necessary. If there are many

independent small requests, an unstriped array can still send different requests to different disks in parallel. This already improves request throughput compared to one disk.

So:

- Striping is very useful for large sequential requests.
- For many small independent requests, parallel disks help even without striping.
- Striping may sometimes reduce request-rate performance for small requests, because one request may involve multiple disks instead of only one.

Conclusion: Striping is not strictly necessary to improve I/O request rate. It mainly improves transfer rate for large requests.

6. RAID Storage Capacity

Given:

4 drives, 200 GB per drive

Raw capacity:

$$4 \times 200 = 800 \text{ GB}$$

RAID Level	Capacity Rule	Available Capacity
RAID 0	All disks store data	$4 \times 200 = 800 \text{ GB}$
RAID 1	Mirroring, about half usable	$\frac{4}{2} \times 200 = 400 \text{ GB}$
RAID 2	Hamming-code ECC disks	200 GB for 4 disks using 1 data + 3
RAID 3	One parity disk	$(4 - 1) \times 200 = 600 \text{ GB}$
RAID 4	One parity disk	$(4 - 1) \times 200 = 600 \text{ GB}$
RAID 5	Distributed parity, one disk equivalent	$(4 - 1) \times 200 = 600 \text{ GB}$
RAID 6	Double distributed parity, two disk equivalents	$(4 - 2) \times 200 = 400 \text{ GB}$

Note: RAID 2 is rarely used in practice. With only 4 disks, Hamming-code protection leaves only 1 data disk and 3 check disks.

2 File Management Review Questions

1. Difference Between a Field and a Record

A **field** is one piece of data, such as a name, ID, or grade. A **record** is a group of related fields that describe one entity, such as one student record.

2. Difference Between a File and a Database

A **file** is a collection of related records stored together. A **database** is an organized collection of related data, usually managed by a DBMS and supporting searching, relationships, security, and consistency.

3. File Management System

A **file management system** is the part of the operating system that creates, stores, organizes, retrieves, protects, and deletes files on storage devices.

4. Typical Directory Operations

Typical directory operations include creating a file, deleting a file, listing directory contents, renaming files, searching for a file, creating subdirectories, deleting subdirectories, and moving files.

5. Pathname and Working Directory

A **pathname** specifies the location of a file. If it is an absolute pathname, it starts from the root directory. If it is a relative pathname, it is interpreted starting from the current **working directory**.

6. Typical File Access Rights

Typical access rights include read, write, execute, append, delete, list, change permissions, and ownership control.

7. Three File Allocation Methods

- **Contiguous allocation:** A file occupies consecutive blocks on disk. It is fast for sequential access but suffers from external fragmentation.
- **Linked allocation:** Each file block points to the next block. It avoids external

fragmentation but is slower for direct access.

- **Indexed allocation:** An index block stores pointers to all file blocks. It supports direct access but needs extra space for the index block.

3 File Management Problems

1. Increasing Portion Size Allocation

The file has:

N records

Blocking factor:

F records per block

So the number of blocks needed by the file is:

$$B = \left\lceil \frac{N}{F} \right\rceil$$

The allocation scheme uses portions of increasing size:

$1, 2, 4, 8, \dots$

If the file has k allocated portions, the total allocated number of blocks is:

$$1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1$$

a. Upper limit on number of file allocation table entries

Each portion needs one entry in the file allocation table. We need enough portions to cover B blocks:

$$2^k - 1 \geq B$$

$$2^k \geq B + 1$$

$$k \geq \log_2(B + 1)$$

Therefore, the upper limit is:

$$k = \left\lceil \log_2 \left(\left\lceil \frac{N}{F} \right\rceil + 1 \right) \right\rceil$$

b. Maximum unused allocated file space

The largest unused space occurs just after a new portion is allocated. If the new portion has size 2^{k-1} blocks, almost all of that new portion may still be unused.

So the unused space is less than one full newly allocated portion:

$$\text{Maximum unused space} < 2^{k-1} \text{ blocks}$$

In records, this is:

$$\text{Maximum unused space} < F \times 2^{k-1} \text{ records}$$

This means the unused allocated space is always less than about half of the total allocated space.

2. Percentage of Wasted Space with 16K Blocks

Block size:

$$16K = 16 \times 1024 = 16384 \text{ bytes}$$

For each file:

$$\text{Number of blocks} = \left\lceil \frac{\text{file size}}{16384} \right\rceil$$

$$\text{Wasted space} = \text{allocated space} - \text{file size}$$

$$\text{Wasted percentage} = \frac{\text{wasted space}}{\text{allocated space}} \times 100$$

File Size	Blocks Needed	Allocated Space	Wasted Space	Waste %
41,600 bytes	$\lceil 41600/16384 \rceil = 3$	49,152	7,552	15.36%
640,000 bytes	$\lceil 640000/16384 \rceil = 40$	655,360	15,360	2.34%
4,064,000 bytes	$\lceil 4064000/16384 \rceil = 249$	4,079,616	15,616	0.38%

3. Advantages of Using Directories

Directories are useful because they:

- Organize files into logical groups.
- Allow different files to have the same name if they are in different directories.
- Make searching and navigation easier.
- Support hierarchical structure using subdirectories.

- Help apply protection and access rights to groups of files.

4. Directories as Special Files vs Ordinary Data Files

Directories as special files:

- **Advantages:** Safer because users cannot freely corrupt directory contents. The OS controls updates to filenames, links, and metadata.
- **Disadvantages:** Less flexible. Users and programs cannot manipulate directories using normal file operations.

Directories as ordinary data files:

- **Advantages:** More flexible and simple conceptually because directories can be read and managed like files.
- **Disadvantages:** More dangerous because incorrect writes may damage the file system structure.

Conclusion: Treating directories as special files is usually safer, while treating them as ordinary files is more flexible but riskier.

5. Limiting the Depth of a Tree-Structured File System

If the OS limits the depth of the directory tree, users cannot create directory paths deeper than that limit.

Effect on users:

- Users have less freedom to organize files into many nested folders.
- Very large projects may become harder to organize clearly.
- Pathnames become shorter and simpler.

Effect on file system design:

- Directory traversal becomes simpler.
- Maximum pathname length is easier to control.
- The OS can use simpler algorithms and smaller fixed-size structures.

- Searching and protection checks may become easier.

Conclusion: The limit makes the file system easier to implement, but it reduces user flexibility.